Herald

Herald — `commons` Module Guide (Operator / Developer)

Field	Value
Revision	1
Created	2026-05-31
Last modified	2026-05-31
Status	active
Status summary	Nano-detail per-module reference for `commons` — Herald's **L0 foundation** layer. It owns the §11.0 `Channel` interface + every value type that crosses an adapter boundary (`OutboundMessage`, `Receipt`, `InboundEvent`, `Subscriber`, `CloudEventEnvelope`, `TraceContext`, `Branding`, `ChannelID`, `Capabilities`, …), the deterministic `Clock` abstraction (`RealClock` / `FakeClock`), the time-ordered `NewUUIDv7` helper, the per-flavor `DefaultBranding` factory + `ProjectName` resolver, and the §11.4.104 Participant identity model (`Participant`, `IdentityResolver` / `MemoryResolver`, `OperatorHandleFromEnv`, the `Claude` sentinel, and the `MentionsFor` tagging matrix). `commons` imports nothing from any other Herald module; every other module imports it. ANTI-BLUFF: every type/function below was read from the real source under `commons/` as of this revision.
Issues	(none specific to this guide)
Continuation	bump when the §11.0 `Channel` contract gains a new method or value type, when the Participant model adds a non-in-memory `IdentityResolver` (DB-backed), or when `DefaultBranding` gains a new flavor key.

Table of contents

---

§1. Overview — the L0 base every module imports

`commons` (Go package `commons`, module path `github.com/vasic-digital/herald/commons`) is Herald's **L0 foundation layer** (spec V4 §10). Its `go.mod` header states the design rule literally:

> Every other Herald module imports from here; commons imports nothing from other Herald modules — keeping it dependency-free at the Herald layer.

It owns the cross-cutting contracts and value types that would otherwise be reinvented per adapter:

- the **§11.0** `Channel` **interface** and its complete set of value types (§2, §3);
- the `Clock` time-source abstraction so nothing outside `clock.go` calls `time.Now()` directly (§4);
- the `NewUUIDv7` time-ordered primary-key generator (§5);
- the **per-flavor** `Branding` **factory** (`DefaultBranding`) + the Claude Code session-name resolver `ProjectName` (§6);
- the **Participant identity model** — `Participant`, `IdentityResolver` / `MemoryResolver`, `OperatorHandleFromEnv`, the reserved `Claude` sentinel, and the `MentionsFor` @-tagging matrix (§7).

At the Herald layer `commons` is dependency-light: per `commons/go.mod` its only Herald-external runtime dependency for these files is `github.com/google/uuid v1.6.0` (used for `uuid.UUID` primary keys and native UUIDv7). The package doc comment in `types.go` enforces the no-reinvention rule:

> Adapters under commons_messaging/channels// consume these types directly; no adapter is allowed to invent its own equivalent (spec §11.0).

§2. The §11.0 Channel contract

`commons/types.go` defines the interface every channel adapter (Telegram, Slack, the `null://` sandbox, …) implements:

```
type Channel interface {
    Name()
        string                                                    //
        "tgram", "slack", ...
    Capabilities()
        Capabilities                                          //
        declarative feature flags
    Send(ctx context.Context, msg OutboundMessage) (Receipt,
        error)
    Subscribe(ctx context.Context, h InboundHandler) error //
        long-running, called by `serve`
    HealthCheck(ctx context.Context) error
}
```

`Capabilities` is the declarative feature-flag struct routers consult **before** dispatching (a mismatch is logged and the message is dropped or downgraded by the adapter):

```
type Capabilities struct {
    Text             bool
    Markdown         bool // adapter's native markdown flavor
    HTML             bool
    Attachments      bool
    AttachmentMaxMiB int
    Threads          bool             // first-class reply
        threading
    InteractiveURL   bool             // URL buttons / link
        actions
    InteractiveCall  bool             // callback handlers
        (advanced tier)
    DeliveryCeiling  DeliveryEvidence // see §17 / R-05
}
```

`DeliveryEvidence` is an ordered enum naming the strongest reachable delivery signal per channel — `DeliveryUnknown`, `DeliveryAccepted` (hop-by-hop ack, SMTP 250), `DeliveryRouted` (platform stored & broadcast — Telegram/Slack API ok), `DeliveryDelivered` (recipient transport confirmed), `DeliveryRead` (recipient read marker). Its `String()` method returns the lowercase spec wording (`"accepted"`, `"routed"`, `"delivered"`, `"read"`, else `"unknown"`).

§3. Core value types

These are the values that cross the `Channel` boundary, all in `commons/types.go`.

§3.1 `OutboundMessage` — what you pass to `Channel.Send`

```
type OutboundMessage struct {
    EventID        string           // CloudEvents id (§4)
    IdempotencyKey string           // explicit; falls back to
        EventID
    TenantID       string           // UUID; matches
        `subscribers.tenant_id`
    To             []Recipient      // resolved from preferences
        + tag fan-out
    Subject        string           // optional (Email, RSS-like
        channels)
    Body           Body             // rendered per-channel
        template output
    Attachments    []Attachment
    Thread         *ConversationRef // optional; per §12
    Priority       Priority         // ntfy-compatible 1..5
    Actions        []Action         // optional interactive
        buttons
    Branding       Branding         // per-flavor (§6.3)
```

```
    Trace          TraceContext     // OTel propagation
}
```

Supporting types:

- `Body` carries multiple rendered representations (`Plain`, `Markdown`, `HTML`, plus `Native map[string]any` for adapter-specific payloads like Slack Block-Kit JSON); the adapter picks the best match for its `Capabilities`.
- `Attachment` holds `Filename`, `MIMEType`, `SizeBytes`, a lazy `Reader func() (io.ReadCloser, error)` (so the adapter streams without buffering the whole payload), and an optional inline-image `CID` (Email).
- `Recipient` is a resolved `{Channel, ChannelUserID, DisplayName}` triple (e.g. `{"tgram", chat_id, …}`).
- `Action` is an interactive-UI hint with an `ActionType` (`ActionView`, `ActionURL`, `ActionCallback`, `ActionHTTP`, `ActionCopy`) plus `Label`/`URL`/`Method`/`Body`/`Data`.
- `Priority` maps to ntfy 1..5: `PriorityLow=1`, `PriorityNormal=3`, `PriorityHigh=4`, `PriorityUrgent=5`.

§3.2 `Receipt` — what `Channel.Send` returns on success

```
type Receipt struct {
    Evidence      DeliveryEvidence
    ChannelMsgID  string // Slack ts; Telegram message_id; SMTP
        queue-id; ...
    SentAt        time.Time
    LatencyMillis int64
    Native
        map[string]any // adapter-specific raw response (for
        diary)
}
```

`Evidence` is the adapter's proof-of-acceptance/routing/delivery that lets the router decide whether to retry.

§3.3 Inbound: `InboundHandler` + `InboundEvent`

`Subscribe` feeds messages to an `InboundHandler`:

```
type InboundHandler interface {
    Handle(ctx context.Context, ev InboundEvent) error
}

type InboundEvent struct {
    EventID     string             // UUIDv7
    CloudEvent  CloudEventEnvelope // §4.1
    Sender      Recipient          // who sent it
    Subscriber  *Subscriber        // resolved via
        subscriber_aliases, nil if unknown
    Body        Body
    Attachments []Attachment
```

```
    Thread      *ConversationRef
    Raw         map[string]any // adapter-specific raw payload
        (for diary)
}
```

`ConversationRef` is the per-channel thread identifier (`Channel ChannelID`, `ThreadID`, `ParentMessageID`, `RootMessageID` — mapped to Slack `thread_ts` / Telegram `message_thread_id` / Email `References`, etc.).

§3.4 `Subscriber` + `SubscriberAlias`

`Subscriber` is the in-memory projection of the `subscribers` row plus its linked `subscriber_aliases`:

```
type Subscriber struct {
    ID          uuid.UUID
    TenantID    uuid.UUID
    Handle      string // empty if not operator-mapped
    DisplayName string
    Locale      string // BCP-47, e.g. "en-US", "sr-Latn-RS"
    Timezone    string // IANA, e.g. "Europe/Belgrade"
    Roles       []string
    Metadata    map[string]any
    Aliases     []SubscriberAlias
    Preferences *PreferenceSet
}
```

`SubscriberAlias` is one `subscriber_aliases` row (`Channel`, `ChannelUserID`, `VerifiedAt`, `LastSeenAt`). The preference types (`PreferenceSet`, `CategoryPref`, `WorkflowPref`, `QuietHours`) model the per-subscriber opt-in/channels/quiet-hours JSON.

§3.5 `CloudEventEnvelope`

Herald's typed projection of a CloudEvents v1.0 payload (the in-process canonical form after parsing/validation of the boundary `cloudevents.Event`):

```
type CloudEventEnvelope struct {
    SpecVersion     string
    ID              string // UUIDv7
    Source          string // URI
    Type            string // reverse-DNS
    Time            time.Time
    Subject         string            // tag:/channel:/empty
    DataContentType string            // e.g. "application/json"
    Data            []byte            // opaque payload
    Extensions      map[string]string // heraldtenant,
        heraldidempotencykey, heraldpriority, ...
}
```

§3.6 TraceContext

W3C Trace-Context / OpenTelemetry propagation across Herald boundaries:

```
type TraceContext struct {
    TraceID    string // 32-hex chars (W3C Trace Context)
    SpanID     string // 16-hex chars (the parent span at
        handoff)
    TraceFlags byte   // sampling flags (W3C)
    TraceState string // vendor-specific (W3C tracestate header)
    Baggage    string // W3C baggage header value
}
```

§3.7 Branding

The per-flavor visual identity threaded through every `OutboundMessage` (see §6 for the factory that populates it):

```
type Branding struct {
    AppName        string // "Project Herald", "System
        Herald", ...
    BinaryName     string // "pherald", "sherald", ...
    IconURL        string // for rich embeds
    AccentColorHex string // "#2C7BE5"
    DefaultFooter  string // "Sent by pherald 1.0 · github.com/
        vasic-digital/Herald"

    Flavor      string // single-letter (or short) flavor key:
        "p", "s", "b", "sc", ...
    Prefix      string // 3-letter prefix per §8.2 (e.g. "PHR",
        "SHR")
    DisplayName string // human-readable display name (typically
        == AppName)
    DefaultPort int    // default HTTP listen port (per-flavor,
        70XXX range)
    Mission
        string // one-line mission statement for --help / about
}
```

§3.8 ChannelID

The canonical channel identifier string (must match the scheme used in `channel_addresses.address_url` and the adapter's registered URL scheme):

```
type ChannelID string

const (
    ChannelTelegram ChannelID = "tgram"
    ChannelMax      ChannelID = "max"
    ChannelSlack    ChannelID = "slack"
```

```
    ChannelDiscord  ChannelID = "discord"
    ChannelTeams    ChannelID = "teams"
    ChannelLark     ChannelID = "lark"
    ChannelWhatsApp ChannelID = "whatsapp"
    ChannelViber    ChannelID = "viber"
    ChannelEmail    ChannelID = "mailto"
    ChannelNtfy     ChannelID = "ntfy"
    ChannelGotify   ChannelID = "gotify"
    ChannelWebhook  ChannelID = "webhook"
    ChannelDiary    ChannelID = "diary"
    ChannelNull     ChannelID = "null" // §11.14 sandbox/no-op
        adapter for tests
)
```

§4. The Clock abstraction

`commons/clock.go` exists so **Herald never calls `time.Now()` directly outside this file** — deterministic time is essential for quiet-hours, batching windows, retry backoff, idempotency TTLs, and escalation chains (spec §3.5).

```
type Clock interface {
    Now() time.Time
    Since(t time.Time) time.Duration
    Sleep(d time.Duration)
    After(d time.Duration) <-chan time.Time
    NewTimer(d time.Duration) Timer
}

type Timer interface {
    C() <-chan time.Time
    Stop() bool
}
```

Two implementations ship:

- `RealClock` — a zero-field struct wrapping the stdlib `time` package; used in production.
- `FakeClock` — the test implementation. `NewFakeClock()` anchors it at a deterministic instant (`2026-05-20 12:00:00 UTC`). `Advance(d)` moves the clock forward and **fires any pending `After`/`Timer` channels whose deadlines the advance crosses**; `Sleep(d)` is implemented as `Advance(d)` (it does NOT actually block).

A process-global `var Default Clock = RealClock{}` is provided. Production `main()` leaves it as `RealClock{}`; tests swap it in `TestMain`.

```
// Production:
now := commons.Default.Now()

// Test (deterministic):
fc := commons.NewFakeClock()
```

```
timer := fc.NewTimer(5 * time.Second)
fc.Advance(5 * time.Second) // timer.C() now fires
```

§5. UUIDv7 primary keys

`commons/uuidv7.go` wraps `google/uuid`'s native UUIDv7. UUIDv7 is time-ordered (its first 48 bits are a Unix-millisecond timestamp), so index inserts append to the rightmost B-tree leaf and stay cache-hot — used everywhere Herald generates primary keys (idempotency keys, dead-letter rows, inbound message ids, …):

```
func NewUUIDv7() (uuid.UUID, error) // production path —
        propagate the error
func MustUUIDv7() uuid.UUID         // panics on the rare
        failure; boot/test fixtures only
```

§6. The per-flavor Branding factory

`commons/branding.go` provides two functions.

§6.1 `DefaultBranding`

```
func DefaultBranding(flavor string, version string) Branding
```

Each `<prefix>herald` binary calls this at startup and threads the returned `Branding` through every `OutboundMessage`; channel adapters use it to render rich-message accents. It is a `switch` on the single/short flavor key that populates `AppName`, `AccentColorHex`, `Prefix`, `DisplayName`, `DefaultPort`, and `Mission`, then sets `Flavor`, `BinaryName = flavor + "herald"`, and `DefaultFooter`. The cases as implemented (per §3.5 Wave 2 alignment):

`flavor`	**AppName**	**Prefix**	**DefaultPort**	**Serving?**
`p`	Project Herald	PHR	24791	yes
`s`	System Herald	SHR	24793	yes
`b`	Build Herald	BHR	0	CLI-only
`d`	Deploy Herald	DHR	0	CLI-only
`a`	Alert Herald	AHR	0	CLI-only
`sc`	Scheduled-audit Herald	SCR	0	CLI-only
`i`	Incident Herald	IHR	24794	yes
`r`	Release Herald	RHR	0	CLI-only
`c`	Constitution Herald	CHR	24792	yes
`qa`	QA Herald	QHR	0	CLI-only
(default)	Herald	HER	0	CLI-only

`DefaultPort = 0` signals "no HTTP serve mode" (CLI-only flavor); serving flavors bind a port in the 247xx range. The two-letter keys (`sc`, `qa`) match the corresponding binary names' prefixes (`scherald`, `qaherald`).

```
b := commons.DefaultBranding("p", "1.0")
// b.BinaryName == "pherald"
// b.DefaultFooter == "Sent by pherald 1.0 · github.com/vasic-
        digital/Herald"
```

§6.2 `ProjectName`

```
func ProjectName() string
```

Resolves the Claude Code session name per the Wave 6 operator-locked decision (2026-05-22), in order: (1) `HERALD_PROJECT_NAME` env var when non-empty after `TrimSpace`; (2) otherwise `filepath.Base(os.Getwd())` — the cwd basename; (3) otherwise (cwd unreadable) the literal `"Herald"`. Every pherald subcommand that touches the Claude Code dispatcher MUST call `commons.ProjectName()` rather than hardcoding `"Herald"`, pinning the session name to the operator's project context.

§7. Participant identity + IdentityResolver + the tagging matrix (§11.4.104)

`commons/participant.go` + `commons/tagging.go` implement the Participant identity model — the load-bearing contract surface other streams (inbound, workflow, channel adapters, storage) code against. The authoritative contract is `docs/design/PARTICIPANT_ATTRIBUTION.md` (§1 identity model, §3 tagging matrix); the Go signatures here match that contract exactly. Inherited from HelixConstitution §11.4.104 per §11.4.35.

§7.1 The `Claude` sentinel

```
const SystemAgentHandle = "Claude"
```

The reserved `created_by` / `assigned_to` sentinel for the system agent. It is **NEVER** @-tagged in any notification (it is the system, not a human participant).

§7.2 `Participant`

A logical Subscriber/User — one person/agent who may carry a different `@username` on every messenger:

```
type Participant struct {
    Handle
        string            // canonical, messenger-neutral (e.g.
        "@milos85vasic" or "Claude")
    DisplayName string
    Kind        string            // "human" | "agent" |
        "service" (spec §7.5)
    Usernames   map[string]string // channel ("tgram","slack",…)
        -> "@username" on that channel
}
```

A participant with no entry in `Usernames` for a channel **cannot** be @-tagged there.

§7.3 `IdentityResolver` + `MemoryResolver`

```
type IdentityResolver interface {
    ResolveSender(channel, channelUserID, username string)
        (handle string)
    UsernameFor(handle, channel string) (username string, ok
        bool)
    OperatorHandle() string
}
```

`ResolveSender` maps an inbound message's sender to a canonical handle (unknown senders resolve to their raw normalized `@username` so first-contact users are still attributable). `UsernameFor` returns the participant's `@username` on a target channel — `ok=false` if the handle is unknown OR the participant has no alias on that channel (you cannot tag someone who is not on that messenger). `OperatorHandle` returns the canonical operator handle.

`MemoryResolver` is the concrete in-memory implementation (the one tests and roster-loaded runtime paths use). Build it with:

```
func NewMemoryResolver(operatorHandle string, participants
        []Participant) *MemoryResolver
```

It indexes participants by canonical handle (`byHandle`) and by composite sender key (`bySenderKey`, keyed both by `channel\x00channelUserID` and `channel\x00@username`, using a NUL separator that never collides with real values). `ResolveSender` matches by `(channel, channelUserID)` first, then by `(channel, @username)`, then falls back to the normalized raw username or the raw `channelUserID`. `AddSenderIndex(channel, channelUserID, handle)` registers a chat/user-id → handle mapping after construction (the `channel_user_id` is distinct from the `@username`). A compile-time assertion `var _ IdentityResolver = (*MemoryResolver)(nil)` guarantees conformance. The private `normalizeUsername` guarantees exactly one leading `@` and trims whitespace.

§7.4 `OperatorHandleFromEnv`

```
func OperatorHandleFromEnv(channel string) string
```

Reads `HERALD_<CHANNEL>_OPERATOR_USERNAME` for the given channel — e.g. `channel="tgram"` → `HERALD_TGRAM_OPERATOR_USERNAME` (uppercased channel, value returned verbatim after trim; empty means "no operator configured for this channel"). Per §1 the Telegram operator username is the operator's canonical handle, so roster-building callers typically pass channel `"tgram"`.

§7.5 The `MentionsFor` tagging matrix

`commons/tagging.go` implements the §3 matrix exactly:

```
func MentionsFor(createdBy, assignedTo, operatorHandle, channel
        string, r IdentityResolver) []string
```

The rule, verbatim from the source's documented pseudocode:

```
mentions = {}
if assigned_to is a human handle AND assigned_to != Operator:   mentions +
if created_by  is a human handle AND created_by  != Operator AND created_b
                                                                mentions +
# "Claude" is NEVER tagged (it is the system).
# Operator is NEVER tagged (no self-ping).
# de-dup; resolve UsernameFor(handle, channel) — skip if not on that chann
```

Behaviour as implemented: the returned slice holds the **canonical handles** (not the resolved `@usernames`) in a stable order — `assigned_to` before `created_by` — de-duplicated. A handle is dropped if it is empty, equals `SystemAgentHandle("Claude")`, equals `operatorHandle`, is a duplicate, or has no alias on `channel` (`r.UsernameFor(handle, channel)` returns `ok=false`); a `nil` resolver yields no mentions. `operatorHandle` is passed explicitly (rather than read from `r`) so callers that already know the operator for a non-primary channel can override — pass `r.OperatorHandle()` for the default.

```
r := commons.NewMemoryResolver(
    commons.OperatorHandleFromEnv("tgram"), // e.g.
        "@milos85vasic"
    roster,
)
mentions := commons.MentionsFor(
    /*createdBy*/ "Claude",
    /*assignedTo*/ "@dev_jane",
    /*operatorHandle*/ r.OperatorHandle(),
    /*channel*/ "tgram",
    r,
)
// "Claude" dropped (system), operator never tagged; mentions ==
        ["@dev_jane"] (canonical handle).
```

§8. How flavors and other modules consume it

`commons` sits at the bottom of the layering described in `CLAUDE.md` ("Layered shared code: `commons` → `commons_messaging` (level 1) → … → flavor"). Concretely:

- **Channel adapters** (`commons_messaging/channels/<name>/` — `null`, `tgram`, …) implement `commons.Channel` and consume `OutboundMessage` / `Receipt` / `InboundEvent` / `Capabilities` directly — they may not invent equivalents (`types.go` package doc).
- **Flavor binaries** (`pherald`, `sherald`, `iherald`, `qaherald`, …) call `commons.DefaultBranding("<flavor>", version)` at startup and `commons.ProjectName()` for the Claude Code session name.
- **Inbound + workflow streams** use the Participant model — `MemoryResolver` for sender resolution and `MentionsFor` to compute outbound @-mentions.
- **Storage / event paths** use `commons.NewUUIDv7()` for primary keys and the `Clock` abstraction (via `commons.Default` or an injected `FakeClock`) instead of `time.Now()`.

Import the module with its module path:

```
import "github.com/vasic-digital/herald/commons"
```

§9. Testing notes

Tests live alongside the source and run with no external services:

```
go test -race -count=1 ./commons/...
```

Test file	Covers
`commons/branding_test.go`	`DefaultBranding` per-flavor fields + `ProjectName` resolution.
`commons/clock_test.go`	`FakeClock` advance / timer-firing determinism.
`commons/participant_test.go`	`MemoryResolver` sender/username resolution + `MentionsFor` tagging-matrix truth table (incl. the NEGATIVE case proving the Operator is not tagged and the `Claude` sentinel is dropped).
`commons/participant_stress_chaos_test.go`	§11.4.85 stress + chaos coverage of the participant/tagging paths.

(The `commons` value types in `types.go` and the `uuidv7.go` helpers are exercised transitively by the consuming modules' tests; `types.go` carries no behaviour of its own beyond the `DeliveryEvidence.String()` enum mapping.)

§10. References

- Source: `commons/types.go`, `commons/clock.go`, `commons/uuidv7.go`, `commons/branding.go`, `commons/participant.go`, `commons/tagging.go` (and their `_test.go` siblings).
- Module doc: the header comment in `commons/go.mod` (the "imports nothing from other Herald modules" L0 rule).
- Spec: `docs/specs/mvp/specification.V4.md` §10 (L0 layering) + §11.0 (the `Channel` contract + value types), §3.5 (`Clock` + per-flavor identity), §4 (CloudEvents), §7 (subscribers / preferences).
- Participant contract: `docs/design/PARTICIPANT_ATTRIBUTION.md` (§1 identity model, §3 tagging matrix) — inherited from HelixConstitution §11.4.104 per §11.4.35.
- Project guidance: `CLAUDE.md` — "End-user-usability covenant" (§107), "Participant identity, attribution & notification-tagging" (§109), and the layered-shared-code convention.

Sources verified

This guide documents internal Herald source only; no external service/library online documentation was relied on. All behavioural claims are grounded in the cited `commons/*.go` source files as read on 2026-05-31.

Verified 2026-05-31: internal doc — no external online sources. Type/function signatures and behaviour derive from `commons/types.go`, `commons/clock.go`, `commons/uuidv7.go`, `commons/branding.go`, `commons/participant.go`, `commons/tagging.go` (read 2026-05-31); the only third-party dependency at this layer is

`github.com/google/uuid v1.6.0`, pinned in `commons/go.mod`. Re-verify on a `commons` API change (new `Channel` method / value type, new `DefaultBranding` flavor, or a non-in-memory `IdentityResolver`).